BitTitan MigrationWiz — M365 → Google Migration Execution

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — BitTitan MigrationWiz: M365 → Google Migration Execution 5

Outcome . . . 5

Approach . . . 5

Why this is low-risk . . . 5

What the buyer receives . . . 6

1. Solution Overview — BitTitan MigrationWiz: M365 → Google Migration Execution 6

What this solution is . . . 6

Why this matters . . . 6

The MigrationWiz model (how the tool works) . . . 7

Where MigrationWiz fits vs native tools . . . 7

Coexistence and mail routing during the transition . . . 7

The named artifacts . . . 8

What this is not . . . 8

The deployment model . . . 9

2. Requirements Guide — BitTitan MigrationWiz: M365 → Google Migration Execution 9

2.1 A MigrationWiz account with the required licensing . . . 9

2.2 Source M365 / Entra access and a source app registration . . . 9

2.3 Google Workspace target: super-admin, service account, and domain-wide delegation . . . 10

2.4 An endpoint design — source and target, proven . . . 10

2.5 A project design — data types, mappings, and the DMA decision . . . 10

2.6 A pre-stage / delta and throttling plan . . . 11

2.7 A pilot wave, per-user validation, and a DNS/MX cutover runbook with rollback . . . 11

Checklist . . . 11

Global Requirements — Google Workspace & Gemini Solutions 12

1. Google Workspace / Cloud Identity tenant . . . 12

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 12

3. DNS control for the routed domains . . . 13

4. Identity & provisioning source of truth . . . 13

5. A pilot scope (OU / group) and change control . . . 13

6. Security, compliance & governance stakeholders . . . 13

3. Implementation Guide — BitTitan MigrationWiz: M365 → Google Migration Execution 14

Step 1 — Assess: baseline the source and target . . . 14

Step 2 — Design: fix the endpoints and projects . . . 14

Step 3 — Configure: build and prove the endpoints, then the projects . . . 15

Step 4 — Configure: author the pre-stage / delta and throttling plan . . . 15

Step 5 — Author the DNS/MX cutover runbook and rollback . . . 16
Step 6 — Pilot: run Wave 0 end to end . . . 16
Step 7 — Validate: fix the runbook, not just the wave . . . 17
Step 8 — Expand wave by wave . . . 17
Step 9 — Decommission coexistence . . . 17
5. Testing Guide — BitTitan MigrationWiz: M365 → Google Migration Execution 18
Test 1 — Endpoint authorization (pre-flight) . . . 18
Test 2 — Line-item mapping integrity . . . 18
Test 3 — Pre-stage completeness . . . 18
Test 4 — Per-user data-type validation (mail / documents / chat) . . . 19
Test 5 — Coexistence mail routing . . . 19
Test 6 — Throttling / provider-limit health . . . 19
Test 7 — Delta pass idempotency (no duplication) . . . 19
Test 8 — DNS/MX cutover and rollback . . . 19
Wave go/no-go gate . . . 20
6. Troubleshooting Guide — BitTitan MigrationWiz: M365 → Google Migration Execution 20
Every line item fails with an authorization/permission error . . . 20
Passes crawl or stall (throttling) . . . 20
Cutover window overruns . . . 21
Items appear duplicated after a re-run . . . 21
Recent mail/items missing after cutover . . . 21
Documents migrate but SharePoint content lands in the wrong place . . . 21
Teams-to-Chat migration is incomplete or unexpected . . . 21
A line item won't resolve (source or target) . . . 22
The Device Management Agent (DMA) was deployed but isn't needed (or is needed but wasn't) . . . 22
Mail is lost or double-delivered during coexistence . . . 22
Validation fails on item counts even though data looks present . . . 22
Where to go for tool-internal issues . . . 22
7. Best Practices — BitTitan MigrationWiz: M365 → Google Migration Execution 23
Prove endpoints before you move anything . . . 23
Pre-stage heavy, delta light . . . 23
Throttle deliberately; treat provider limits as real . . . 23
One frozen design, run every wave . . . 23
Validate per user, across every data type . . . 23
Make the DMA a deliberate decision, not a reflex . . . 23
Design coexistence up front; decommission it on purpose . . . 24

Every production change is reversible 24
Stay inside the vendor boundary 24
Keep credentials out of code and least-privilege 24
8. Deliverables Summary — BitTitan MigrationWiz: M365 → Google Migration Execution 24
Documentation 24
Named artifacts 25
Python tools 26
The execution playbook (outline) 26
What the buyer receives 27
Outcome 27
Appendix A — PowerShell Scripts 28

Executive Summary — BitTitan MigrationWiz: M365 → Google Migration Execution

Outcome

Every in-scope user moved from Microsoft 365 to Google Workspace on **BitTitan MigrationWiz** — mail, calendar, and contacts in Gmail; OneDrive and SharePoint content in Google Drive and Shared Drives; Teams collaboration in Google Chat — with each user **validated across every data type**, the mail-flow cutover **governed and reversible**, and the whole migration **proven on a pilot wave** before any production wave. Migrations run predictably at enterprise scale, without the endpoint failures, throttling stalls, and unvalidated cutovers that derail tool-driven moves.

Approach

The solution is the **MigrationWiz execution playbook**. It stands up and **proves the endpoints** (an M365 source backed by an Entra app registration, a Google target backed by a service account with domain-wide delegation) before any bulk move; designs the **mail, document, and Teams-to-Chat projects** and their mappings once and freezes them; runs a heavy **pre-stage pass** ahead of cutover so only a small **delta** remains at the switch; sets **throttling** to stay within provider limits; and **validates each user** before the DNS/MX cutover — then expands **wave by wave** behind explicit go/no-go gates. Five Python tools automate the read-only assessment, the line-item and validation CSV artifacts MigrationWiz consumes and produces, the wave/pre-stage/delta plan, the throttling health check, and the target-delegation config check. The house model applies throughout: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Why this is low-risk

- **Pilot-first, gated, reversible.** Nothing reaches production before a named pilot wave proves the full runbook end to end; every production change (mail flow, DNS, cutover) runs under change control with a documented, tested rollback; a wave that fails validation is stopped by the gate.
- **Endpoints proven before scale.** Authorization failures are caught on one test line item, not discovered across thousands of users.
- **Pre-stage/delta keeps cutovers short.** The bulk moves ahead of time; the cutover moves only the delta — validated as non-duplicating — so cutover windows are predictable.
- **Validated per user, per data type.** No user is "done" on mail alone; mail, documents, and chat are each confirmed.
- **Inside the vendor boundary.** MigrationWiz console internals, licensing, and support are deferred to BitTitan's own documentation and flagged "verify for your version"; the tools drive the migration via the documented CSV/validation artifacts, never an invented API. Google and Microsoft configuration is authored at exact, current detail.

What the buyer receives

The full implementation, requirements, testing, troubleshooting, and best-practices guides; five Python automation tools (read-only or dry-run by default, no credentials in code); and a repeatable wave-based execution runbook that turns an M365 → Google MigrationWiz move into a predictable, low-risk program — at a fraction of bespoke consulting cost.

1. Solution Overview — BitTitan MigrationWiz: M365 → Google Migration Execution

What this solution is

Many enterprises run their Microsoft 365 → Google Workspace move on **BitTitan MigrationWiz** — a SaaS migration platform that moves **mailboxes, documents, and collaboration data** from a source tenant to a target tenant over the vendors' own APIs. This solution is the **MigrationWiz execution playbook**: it sets up the **project and endpoints** (the M365 source and the Google Workspace target, each backed by a real app registration / service identity), deploys the **Device Management Agent (DMA)** only where a local-source path requires it, configures the **mail**, **document**, and **Teams-to-Chat** migration projects, runs **pre-stage** and **delta** passes with **throttling**, and **validates** per user — executed **pilot-first** and **rollback-ready** so migrations run predictably at enterprise scale.

It is authored at the **procedural playbook level**: architecture, prerequisites, endpoint and project design, wave / pre-stage / delta sequencing, throttling, and validation. The **MigrationWiz console internals** — exact screen labels, the deep behavior of each project option, licensing (User Migration Bundle vs per-migration licenses), and support — are **deferred to BitTitan's own documentation** and flagged **"verify for your version"** where console flow or version-specific behavior matters. The Google Admin console, Google Cloud service-account/OAuth setup, and the Microsoft Entra app-registration side are given at exact, current detail because they are the parts the operator configures directly.

Why this matters

Tool-driven migrations fail for predictable, avoidable reasons. The **endpoints are misconfigured** — an Entra app registration missing a consented permission, a Google service account without domain-wide delegation for the right scopes — so every item fails with an opaque authorization error discovered at scale instead of on one test mailbox. **Throttling is ignored**, so the migration either crawls (source/target API limits throttle it) or trips protective limits and stalls. There is **no pre-stage**, so the entire mailbox and Drive corpus is attempted in the cutover window and the cutover overruns. The **delta pass is misunderstood** — re-running is feared to duplicate items when, run correctly, a delta pass brings only what changed since the last pass. And **nobody validates per user**, so a mailbox is declared "migrated" while its recent mail, calendar, or Drive permissions never arrived.

A disciplined approach fixes the cause. It stands up and **proves the endpoints** before any bulk move, designs the **project structure** (which data types, which mappings) once, runs a heavy **pre-stage pass** well ahead of cutover so only a small **delta** remains at the switch, sets **throttling** deliberately, and **validates each user** across every data type — and only then cuts over and expands by wave. MigrationWiz's own model is built for exactly this: **pre-stage (Pre-Stage Migration) then a delta/full pass at cutover**, per-item **retry**, and **endpoints** that connect each source/target with the credentials and app registrations the vendors require. Where this solution touches vendor-specific behavior, it cites BitTitan's public documentation and flags version-dependent detail for verification, rather than re-documenting the

console.

The MigrationWiz model (how the tool works)

MigrationWiz organizes a migration around a small set of objects. This solution designs and names each one; the console mechanics of creating them are the vendor's to document.

- **MSPComplete / MigrationWiz workspace** — the account/customer context the migration runs in.
- **Endpoints** — a reusable connection to a source or target system (Microsoft 365, Google

Workspace, and, for some paths, a document/OneDrive/SharePoint or Teams source). Each endpoint holds the connection type and the credentials / app-registration or service-account identity the vendor requires. This solution creates a **source M365 endpoint** and a **target Google Workspace endpoint** per project family.

- **Projects** — a migration of one **data type** for a set of line items: a **Mailbox** project

(mail, calendar, contacts), a **Document/Personal-Drive** project (OneDrive → Drive, SharePoint → Shared Drives), and a **Collaboration/Teams** project (Teams → Google Chat/Spaces, "verify for your version"). Each project references a source and target endpoint.

- **Line items (users / migration pairs)** — the source→target mappings a project migrates, one row

per user or resource. This solution generates these from the directory as a **CSV the tool consumes** — it does not invent a private MigrationWiz API.

- **Passes** — how a project runs: a **Pre-Stage Migration** (bulk-move historical data ahead of

cutover, typically bounded by an age filter such as "older than N days"), then a **Full/Delta Migration** at cutover that moves only what remains. Re-runnable; per-item **retry** handles transient failures.

- **Device Management Agent (DMA)** — deployed to endpoints **only** where a project path needs a

local agent (for example certain document/desktop-source scenarios). Cloud-to-cloud mail projects typically do not require it; **verify for your version and project type** which paths need the DMA before planning a deployment.

Where MigrationWiz fits vs native tools

MigrationWiz is one of several supported patterns for an M365 → Google move; this solution governs its execution. Google-native paths — the Admin-console **data migration service** and the desktop **GWMME** — and other third-party tools each have their own solution. Choose MigrationWiz when its throughput, coverage of mail **and** documents **and** collaboration in one platform, and per-item retry/reporting fit the estate. The **wave, pre-stage/delta, coexistence, and validation discipline** in this solution is the same regardless of tool; only the tool's own configuration differs, and that configuration is deferred to BitTitan.

Coexistence and mail routing during the transition

While part of the estate is on Gmail and part is still on Exchange Online, **inbound mail must reach each user's current platform** and calendar free/busy should resolve across the divide. MigrationWiz moves the mailbox contents; the **mail-flow coexistence** is a Google/DNS concern designed alongside:

- **Split delivery** — point the domain **MX at Google**; Gmail receives inbound mail and **routes

onward to Exchange Online** the messages for users not yet migrated, per a routing rule keyed to the migrated population ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery)).

- **Dual delivery** — deliver a copy to both systems, for archiving or a phased overlap

([Google — dual delivery](https://support.google.com/a/answer/9228551)).

Lower the MX TTL (for example to 300 seconds) ahead of cutover so routing changes take effect quickly and a back-out is fast. Coexistence is **transitional** — decommissioned once the estate is fully on Gmail.

The named artifacts

Author these once and reuse the exact names across every project and wave, so the migration stays legible across owners:

Artifact	Name
Source/target scope & readiness baseline	`BTW Discovery – Source & Target – Baseline`
M365 source endpoint	`BTW Endpoint – M365 Source – Prod`
Google Workspace target endpoint	`BTW Endpoint – Google Target – Prod`
Mailbox migration project	`BTW Project – Mailbox – Prod`
Document migration project	`BTW Project – Documents – Prod`
Teams-to-Chat migration project	`BTW Project – Teams to Chat – Prod`
Line-item / user-mapping file	`BTW Mapping – User Pairs – Per Project`
Pilot wave	`BTW Wave – Pilot – Wave 0`
Per-wave production wave	`BTW Wave – <Segment> – Per Wave`
Pre-stage pass	`BTW Pass – Pre-Stage – Per Wave`
Delta / cutover pass	`BTW Pass – Delta Cutover – Per Wave`
Throttling profile	`BTW Throttle – Project Concurrency – Per Wave`
Per-user validation record	`BTW Validation – Mail/Docs/Chat – Per User`
DNS/MX cutover runbook	`BTW Cutover – DNS/MX – Per Wave`

What this is not

This solution **executes a MigrationWiz migration** and governs the wave/pre-stage/delta sequencing, throttling, coexistence, and validation around it. It does **not** re-document MigrationWiz's console screens, project-option internals, licensing, or support (deferred to BitTitan). It does not own the program-level segmentation across mail/Drive/chat (that is the migration-program solution), and it does not replace the tenant-decommission solution. The included Python tools read the **Google Workspace** target (Admin SDK) and the **Microsoft 365 / Entra** source (Microsoft Graph) **read-only**, and **generate the mapping/validation CSV artifacts MigrationWiz consumes and produces** — they call **no** private BitTitan API and move no data themselves.

The deployment model

> *Baseline the source and target and design the endpoints/projects once, prove the whole migration (endpoints → project → pre-stage → delta → validate) on a named pilot wave, validate each user across mail **and** documents **and** chat against the gate, then cut over and expand wave by wave — the repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented, tested rollback for every production change (mail flow, DNS, sharing, cutover).*

2. Requirements Guide — BitTitan MigrationWiz: M365 → Google Migration Execution

Verify every requirement before running the migration. Each entry gives **Description**, **Why required**, **How to verify**, and **How to implement if missing**. Category-wide prerequisites — the Google Workspace target tenant and super-admin, the source M365 / Entra environment, DNS control, the identity source of truth, a pilot scope, and governance stakeholders — are in `global-requirements-google.md`; do not repeat them here. This guide adds what a **MigrationWiz** execution specifically needs. **MigrationWiz account setup, licensing, and console configuration are deferred to BitTitan's own documentation** ("verify for your version"); this guide states what must be true for the tool to run, not how to click through its console.

2.1 A MigrationWiz account with the required licensing

Description: An active **BitTitan / MigrationWiz (MSPComplete)** account for the migration, with migration **licensing** sufficient for the in-scope users and data types — for example a **User Migration Bundle** per user (covering mailbox + document + collaboration for that user) or the per-migration licenses the project types require. The exact license SKUs, quantities, and entitlements are **BitTitan's to define — verify for your version**. Why required: MigrationWiz will not run a paid pass without applied licenses; under-licensing stalls a wave mid-cutover. How to verify: Sign in to the MigrationWiz/MSPComplete workspace and confirm the account is active and that license count ≥ the wave's user count for each data type in scope. Confirm which license type each project (Mailbox / Document / Teams-to-Chat) consumes per BitTitan's documentation. How to implement if missing: Purchase/apply the appropriate MigrationWiz licenses through BitTitan or an authorized reseller before scheduling the pilot. (Licensing detail deferred to BitTitan.)

2.2 Source M365 / Entra access and a source app registration

Description: Administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** sufficient to (a) create/authorize the **app registration** MigrationWiz uses for the source endpoint, and (b) grant the mailbox/OneDrive/SharePoint/Teams access the chosen project types require. For mail, this is typically an app registration with the Graph/Exchange application permissions MigrationWiz's documentation specifies (application access to mailboxes), consented tenant-wide. Why required: The source endpoint reads each mailbox, drive, and collaboration space from M365; without an authorized source identity every line item fails to authenticate. How to verify: Confirm Global Administrator (or the least-privilege roles BitTitan names) in the Entra admin center; confirm you can register an app and grant admin consent. Run `btw-readiness-assessment.py` against the source to confirm you can read the estate over Microsoft

Graph. How to implement if missing: Obtain the Entra role, create the source app registration and grant the application permissions/admin consent MigrationWiz's source-endpoint documentation lists (deferred — verify for your version), and record the app (client) ID in the endpoint design.

2.3 Google Workspace target: super-admin, service account, and domain-wide delegation

Description: A provisioned **Google Workspace** target with a **super admin**, target accounts provisioned for the wave (per the identity source of truth in the global requirements), and a **Google Cloud service account** (or OAuth app) with **domain-wide delegation** authorized in the Admin console for the API scopes MigrationWiz's target endpoint requires (Gmail / Drive / Calendar / Directory as applicable). The automation tools in this solution also use a service account or ADC with **read-only** Admin SDK / Directory / Reports scopes. Why required: The target endpoint writes mail/Drive/Chat into licensed Google accounts using the delegated service identity; a mailbox cannot migrate into an account that does not exist or without delegated API access. How to verify: In **admin.google.com** → **Security** → **API controls** → **Domain-wide delegation**, confirm the service account's **client ID** is authorized for the required scopes; confirm Gmail/Drive licenses for a sample wave; confirm the target accounts are provisioned. How to implement if missing: Provision licenses and target accounts (GCDS/SCIM per the global requirements), create the service account in Google Cloud, and authorize its client ID for the exact scopes MigrationWiz's target-endpoint documentation specifies (deferred — verify for your version) before scheduling the pilot.

2.4 An endpoint design — source and target, proven

Description: A decision record — the endpoint design — fixing the **M365 source endpoint** (`BTW Endpoint – M365 Source – Prod`) and the **Google Workspace target endpoint** (`BTW Endpoint – Google Target – Prod`): connection type, the app-registration / service-account identity each uses, and the scopes/permissions consented. Each endpoint is **proven** by migrating one test line item end to end before any bulk pass. Why required: Endpoints are reused by every project and wave; a misconfigured endpoint fails every line item. Deciding and proving them once prevents authorization failures discovered at scale. How to verify: The design names both endpoints, their identities, and their scopes; one test user migrates a small set of items source→target without an authorization error. How to implement if missing: Create both endpoints in MigrationWiz (console flow deferred to BitTitan), record their identities/scopes in the design, and prove one test line item before the pilot wave.

2.5 A project design — data types, mappings, and the DMA decision

Description: A project design fixing the **projects** to run — `BTW Project – Mailbox – Prod`, `BTW Project – Documents – Prod`, `BTW Project – Teams to Chat – Prod` — the **data types** each moves and in what order, the **mappings** each needs (SharePoint site → Shared Drive, Team → Chat space, resource/room calendars, address rewriting), and an explicit **Device Management Agent (DMA) decision**: whether any in-scope project path requires the DMA and, if so, where it is deployed. (**Verify for your version** which project types need the DMA; cloud-to-cloud mail typically does not.) Why required: Every wave must run the *same* projects, data-type order, and mappings; deciding per wave guarantees rework and inconsistent outcomes. An unplanned DMA requirement discovered mid-wave stalls document migration. How to verify: The design names each project, its data types and order, the required mappings, and the DMA decision, and is agreed by the migration's technical owners.
`btw-project-mapping-builder.py` produces the line-item CSV each project consumes. How to

implement if missing: Hold Implementation Step 2 and freeze the project design before the pilot; generate the mapping CSVs with the builder tool.

2.6 A pre-stage / delta and throttling plan

Description: A pass plan fixing, per project: the **Pre-Stage Migration** (bulk-move historical data ahead of cutover, bounded by an age filter such as "older than N days"), the **Delta/Full pass** at cutover, and a **throttling** profile (`BTW Throttle – Project Concurrency – Per Wave`) that keeps concurrency within source (M365/Graph) and target (Google API) limits so passes neither crawl nor trip protective limits. Provider-side throttling behavior and MigrationWiz's own concurrency controls are **verify-for-your-version**. Why required: Without a pre-stage, the full corpus is attempted in the cutover window and the cutover overruns; without deliberate throttling, passes stall on provider limits. The delta pass is what lets a re-run move only what changed since pre-stage without re-moving migrated items. How to verify: The plan states the pre-stage age filter, the delta timing, and the concurrency/ throttle settings; `btw-wave-planner.py` produces the wave/pass schedule and throttling guidance. How to implement if missing: Author the pass plan in Implementation Step 4; run a pre-stage on the pilot wave and confirm only a small delta remains at cutover.

2.7 A pilot wave, per-user validation, and a DNS/MX cutover runbook with rollback

Description: A named pilot wave — `BTW Wave – Pilot – Wave 0` — of a small, representative user set (core IT and early adopters, plus at least one shared mailbox, one SharePoint site, and one Team so those paths are exercised); a per-user validation record `BTW Validation – Mail/Docs/Chat – Per User` confirming **every** in-scope data type landed; and a repeatable **DNS/MX cutover runbook** (`BTW Cutover – DNS/MX – Per Wave`) applied under **change control** with a documented, tested back-out. Why required: The full migration and cutover must be proven on a pilot before any production wave; each migrated user must be validated across all data types, not declared done on mail alone; and a DNS/MX change is high-impact and must be reversible. How to verify: The pilot wave and validation record exist and cover every data type; the runbook has all stages and a rollback; the change-control path and lowered MX TTL are in place. Build the wave with `btw-project-mapping-builder.py` and validate with `btw-postmigration-validation.py`. How to implement if missing: Define the pilot wave, author the cutover runbook (Step 5), run it end to end (Step 6), and validate per user before expanding.

Checklist

- MigrationWiz account active + licensing ≥ wave (verify for your version) — §2.1
- Source M365/Entra access + source app registration consented — §2.2 · `btw-readiness-assessment.py`
- Google target super-admin + service account with domain-wide delegation — §2.3 · `btw-config-helper.py`
- Endpoint design proven (source + target, one test line item) — `BTW Endpoint – M365 Source – Prod` / `BTW Endpoint – Google Target – Prod`
- Project design frozen (data types, mappings, DMA decision) — `btw-project-mapping-builder.py`

• Pre-stage/delta + throttling plan — `btw-wave-planner.py`

• Pilot wave + per-user validation + DNS/MX cutover runbook w/ rollback — `btw-postmigration-validation.py`

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — BitTitan MigrationWiz: M365 → Google Migration Execution

Follow these steps in order. This guide executes an **M365 → Google Workspace migration on BitTitan MigrationWiz** — proving the endpoints, projects, pre-stage/delta passes, throttling, and DNS/MX cutover on a **named pilot wave**, validating each user across mail, documents, and chat, then expanding **wave by wave**. Artifact names are fixed — create them with the exact names shown and reuse them every wave. **MigrationWiz console internals** (the exact screens for creating endpoints, projects, passes, and the DMA installer, plus licensing) are **deferred to BitTitan's documentation**; this guide governs the repeatable procedure and sequencing around them and flags version-dependent behavior **"verify for your version."**

> *Model: baseline and design once, prove the full migration and cutover on a pilot wave, validate mail/docs/chat per user against the gate, then cut over DNS/MX and expand. The repeatable way: **Assess → Design → Configure → Pilot → Validate → Expand.***

Step 1 — Assess: baseline the source and target

1. Run `btw-readiness-assessment.py` to baseline both sides read-only: the **M365 source** (users, licensed mailboxes, shared mailboxes, resource/room mailboxes, OneDrive/SharePoint sites, Teams — read over Microsoft Graph) and the **Google target** (users provisioned, licenses, and whether the service account's domain-wide delegation is present — read over the Admin SDK). Record it as `BTW Discovery – Source & Target – Baseline`.

1. Add the details the APIs cannot read: per-mailbox and per-site **data volumes**, SharePoint site → Shared Drive intentions, Team → Chat-space intentions, resource-calendar mappings, and any mail-touching integrations (relay connectors, forwarding, shared-mailbox delegation) that affect the migration or routing.

1. Record gaps against the Requirements checklist. **Do not proceed** past missing source app registration, missing target delegation, under-licensing, or a missing identity source of truth — resolve those first.

Step 2 — Design: fix the endpoints and projects

1. **Endpoint design.** Fix the two reusable endpoints and the identity each uses:

• `BTW Endpoint – M365 Source – Prod` — the source connection using the Entra **app registration**

from §2.2 (client ID + the application permissions/admin consent MigrationWiz's source-endpoint docs specify; **verify for your version**).

- `BTW Endpoint – Google Target – Prod` — the target connection using the Google **service account**

with domain-wide delegation from §2.3 (the scopes MigrationWiz's target-endpoint docs specify; **verify for your version**).

1. **Project design.** Fix which projects run, their data-type order, and mappings:

- `BTW Project – Mailbox – Prod` — mail, calendar, contacts (Exchange Online → Gmail/Calendar/

Contacts).

- `BTW Project – Documents – Prod` — OneDrive → user Drive, SharePoint sites → **Shared Drives**

(record each site→drive mapping).

- `BTW Project – Teams to Chat – Prod` — Teams → Google Chat/Spaces (record each Team→space mapping;

verify for your version which collaboration data types are in scope).

1. **DMA decision.** Decide explicitly whether any in-scope project path requires the **Device

Management Agent**. Cloud-to-cloud mail typically does not; some document/desktop-source paths do —** verify for your version and project type**. If required, record where the DMA is deployed; if not, record "no DMA required" so no one deploys it speculatively.

1. Generate the line-item **mapping CSVs** each project consumes with `btw-project-mapping-builder.py`

(reads the source directory + target directory, emits `BTW Mapping – User Pairs – Per Project` with validated source→target address pairs). **Freeze** the design before the pilot; every wave runs it unchanged.

Step 3 — Configure: build and prove the endpoints, then the projects

1. Create both endpoints in MigrationWiz per the design (console flow deferred to BitTitan). Run

`btw-config-helper.py --check-delegation` to confirm the Google service account's client ID is authorized for the required scopes and the source app registration is reachable, before proceeding.

1. **Prove the endpoints** on **one test line item**: migrate a small set of mail/docs for a single

test user source→target and confirm no authorization error. Endpoints are a prerequisite, not a pilot step.

1. Create the three projects, attaching the source and target endpoints and importing the pilot wave's

mapping CSV as line items. Set the project options (data-type filters, address mapping, folder/label mapping) per the design; **exact option behavior is verify-for-your-version.**

Step 4 — Configure: author the pre-stage / delta and throttling plan

1. Author the pass plan per project with `btw-wave-planner.py`:

• **Pre-Stage pass** `BTW Pass – Pre-Stage – Per Wave` — a bulk pre-stage migration bounded by an

age filter (e.g. "items older than N days") run well ahead of cutover, so only a small delta remains at the switch.

• **Delta/cutover pass** `BTW Pass – Delta Cutover – Per Wave` — the pass at cutover that moves only

what changed since pre-stage, without re-moving migrated items.

• **Throttling** `BTW Throttle – Project Concurrency – Per Wave` — the concurrency/ item-per-hour

settings that keep passes within source (M365/Graph) and target (Google API) limits. The planner emits throttling **guidance**; the exact MigrationWiz concurrency controls and provider limits are **verify-for-your-version.**

1. Fix the pre-stage age filter, the delta timing relative to the DNS/MX cutover, and the concurrency

ceiling. This plan is applied unchanged every wave.

Step 5 — Author the DNS/MX cutover runbook and rollback

1. Author `BTW Cutover – DNS/MX – Per Wave` with the fixed sequence:

2. **Readiness** — the wave's target accounts are provisioned and licensed, both endpoints prove

one test line item, `btw-config-helper.py` reports delegation/consent healthy, and `btw-config-helper.py --check-throttle` shows no active provider throttling.

1. **Pre-stage** — run `BTW Pass – Pre-Stage – Per Wave` for each project ahead of cutover.

2. **Coexistence routing** — apply **split/dual delivery** (MX at Google, Gmail routes not-yet-

migrated users onward to Exchange Online) under change control, with the MX TTL already lowered.

1. **Delta cutover** — run `BTW Pass – Delta Cutover – Per Wave` to finalize recent mail/docs/chat.

2. **Validate** — run `btw-postmigration-validation.py` per user (mail **and** documents **and**

chat) and confirm the wave passes the gate.

1. **DNS/MX cutover** — apply the staged MX/DNS change for the wave with a documented back-out.

2. **Rollback** — revert routing/DNS to the prior platform and re-point the wave; used only if the

gate fails.

1. Define the go/no-go: the wave passes only when readiness held, the pre-stage completed, all data

types validated per user, the delta pass is clean, and the DNS/MX change is applied and reversible.

Step 6 — Pilot: run Wave 0 end to end

1. Assemble `BTW Wave – Pilot – Wave 0`: a small, representative set (core IT and early adopters) that

includes at least one shared mailbox, one SharePoint site, and one Team so those paths are exercised. Generate its mapping CSV with `btw-project-mapping-builder.py`.

1. Execute the full runbook on the pilot wave: readiness → pre-stage → coexistence routing → delta cutover → validate → DNS/MX cutover.

1. Run `btw-postmigration-validation.py` per user for coverage/exceptions. The pilot passes **only**

when the whole runbook ran end to end and every user validated across every in-scope data type.

Step 7 — Validate: fix the runbook, not just the wave

1. Complete the Testing Guide for the pilot. Treat every issue as a **runbook/design** defect: fix the endpoint identity/scopes, the project mappings, the pre-stage age filter, or the throttling profile so later waves inherit the fix.

1. Re-run the affected pass and validation. Where line items failed on transient errors, use MigrationWiz's per-item **retry** (verify for your version) and re-validate; regenerate the mapping CSV with `btw-project-mapping-builder.py` if source→target pairs changed.

Step 8 — Expand wave by wave

1. For each subsequent wave `BTW Wave – <Segment> – Per Wave`, re-run the same runbook: generate the wave's mapping CSV, pre-stage, apply coexistence routing, delta-cutover, validate per user (`btw-postmigration-validation.py`), and cut over DNS/MX — passing the go/no-go before the next wave. Check provider throttling with `btw-config-helper.py --check-throttle` before each pre-stage.

1. Keep the coexistence routing rule current as the migrated population grows so every user's mail always reaches their current platform; a stale rule silently mis-delivers.

Step 9 — Decommission coexistence

1. After the **final** wave validates and cuts over, retire the split/dual-delivery routing and the onward route to Exchange Online, and finalize the MX so all mail is delivered in Gmail — never mid-migration. Confirm no forwarding loop or stale route remains, then release the MigrationWiz projects/endpoints per BitTitan's guidance.

Order of operations

```
Assess/baseline -> Design (endpoints+projects+DMA) -> Configure/prove
endpoints+projects -> Pre-stage/delta+throttle plan -> Cutover
runbook+rollback -> Wave 0 pilot -> Validate/fix runbook -> Expand wave by
wave -> Decommission coexistence
```

5. Testing Guide — BitTitan MigrationWiz: M365 → Google Migration Execution

Run these validations for **every wave** before its DNS/MX cutover and again after. Each test states what to check and the **expected pass/fail**. A wave passes its go/no-go gate only when all applicable tests pass. Validation is per user across **every in-scope data type** — a user is never "migrated" on mail alone. Use `btw-postmigration-validation.py` (read-only) to drive the per-user checks and `btw-config-helper.py --check-throttle` for provider-limit signals; MigrationWiz's own per-project statistics (items migrated / failed / skipped) are read from the console (**verify for your version**).

Test 1 — Endpoint authorization (pre-flight)

- **Check:** One test line item migrates a small set of items over each endpoint without an

authorization error; `btw-config-helper.py --check-delegation` reports the Google service account's client ID authorized for the required scopes and the source app registration reachable.

- **Pass:** Test item lands source→target; delegation and source consent both healthy.
- **Fail:** Any authorization / permission error — resolve the app registration (source) or

domain-wide delegation scopes (target) before any bulk pass. **Do not run pre-stage on a failed endpoint.**

Test 2 — Line-item mapping integrity

- **Check:** `btw-project-mapping-builder.py --validate` confirms every line item in the wave's CSV has

a resolvable source UPN, a provisioned + licensed target account, and no duplicate/blank rows.

- **Pass:** 100% of line items resolve to a provisioned, licensed target; zero duplicates/blanks.
- **Fail:** Any unresolved source, unprovisioned/unlicensed target, or duplicate — fix the mapping and

re-import before pre-stage.

Test 3 — Pre-stage completeness

- **Check:** After `BTW Pass – Pre-Stage – Per Wave`, MigrationWiz reports the pre-stage pass complete

for each project with failures triaged; the residual (items newer than the age filter) is small and expected.

- **Pass:** Pre-stage complete per project; only the expected recent-item delta remains; failures are

triaged (retryable vs blocked), not ignored.

- **Fail:** Pre-stage stalled, a large unexplained residual, or untriaged failures — investigate

throttling (Test 6) and per-item errors before cutover.

Test 4 — Per-user data-type validation (mail / documents / chat)

• **Check:** `btw-postmigration-validation.py` confirms, per user: the target mailbox exists and mail/

calendar/contacts are present (folder/label and recent-item spot checks); OneDrive/SharePoint content is present in the user Drive / mapped Shared Drive with expected item counts; Teams content is present in the mapped Chat space (**Teams-to-Chat coverage: verify for your version**).

• **Pass:** Every user shows all in-scope data types present; item-count deltas within the tolerance

the design sets.

• **Fail:** Any user missing a data type or with an out-of-tolerance count — re-queue/retry that line

item, re-run the delta pass, and re-validate before declaring the user done.

Test 5 — Coexistence mail routing

• **Check:** With split/dual delivery applied, a test message to a **migrated** user is delivered in

Gmail, and a test message to a **not-yet-migrated** user is routed onward to Exchange Online; a cross-platform meeting resolves free/busy.

• **Pass:** Each test message reaches the correct current platform; no double-delivery or loss;

free/busy resolves across the divide.

• **Fail:** Mis-delivery, double-delivery, or a routing loop — correct the routing rule (keyed to the

migrated population) before the DNS/MX cutover.

Test 6 — Throttling / provider-limit health

• **Check:** `btw-config-helper.py --check-throttle` reads target Google API signals (and source Graph

indicators where available) for throttling/429-style backoff during passes; the throttling profile `BTW Throttle – Project Concurrency – Per Wave` keeps concurrency within limits.

• **Pass:** No sustained provider throttling; passes progress at the planned rate.

• **Fail:** Sustained backoff/throttling or a stalled pass — lower concurrency per the planner's

guidance and re-run; **verify the provider's current limits for your tenant.**

Test 7 — Delta pass idempotency (no duplication)

• **Check:** After the delta/cutover pass, re-validate a sample: migrated items are **not duplicated**

and only items changed since pre-stage were moved.

• **Pass:** No duplicate mail/files; recent items present exactly once.

• **Fail:** Duplicates or missing recent items — review the pass configuration (delta vs full) and the

age filter; **verify pass behavior for your version** before expanding.

Test 8 — DNS/MX cutover and rollback

• **Check:** The staged MX/DNS change applies for the wave and the documented back-out has been tested;

post-cutover, inbound mail for the wave is delivered in Gmail.

• **Pass:** MX/DNS change applied under change control, mail delivered in Gmail, and a proven rollback exists.

• **Fail:** Mail disruption or no tested back-out — invoke rollback, restore prior routing, and do not proceed until the runbook and back-out are proven.

Wave go/no-go gate

A wave passes only when: endpoints authorized (T1), mappings clean (T2), pre-stage complete (T3), **every user validated across every in-scope data type** (T4), coexistence routing correct (T5), no sustained throttling (T6), the delta pass introduced no duplication (T7), and the DNS/MX cutover is applied and reversible (T8). Any fail blocks the wave until remediated and re-tested.

6. Troubleshooting Guide — BitTitan MigrationWiz: M365 → Google Migration Execution

Common failure modes and resolutions when executing an M365 → Google migration on MigrationWiz. Each entry gives the **symptom**, the **likely cause**, and the **resolution**. Deep MigrationWiz console internals, per-error-code behavior, licensing, and support are **BitTitan's to document** ("verify for your version"); this guide covers the procedure-level failures the operator resolves and points to the right vendor surface for the rest.

Every line item fails with an authorization/permission error

• **Likely cause:** An endpoint identity is wrong — the source Entra **app registration** is missing a consented application permission, or the Google **service account** lacks **domain-wide delegation** for the required scopes. A whole-project identical failure almost always means the endpoint, not the data.

• **Resolution:** Run `btw-config-helper.py --check-delegation`. Fix the source app registration (permissions + admin consent) per MigrationWiz's source-endpoint docs, and authorize the SA client ID for the exact scopes in **Admin console** → **Security** → **API controls** → **Domain-wide delegation**. Re-prove one test line item before re-running any pass. Do not run pre-stage on a failed endpoint.

Passes crawl or stall (throttling)

• **Likely cause:** The pass tripped a source (M365/Graph) or target (Google API) protective limit and is backing off; concurrency is set too high, so more threads make it worse.

• **Resolution:** Run `btw-config-helper.py --check-throttle`. If throttling is sustained, **lower**

project concurrency per `btw-wave-planner.py` guidance and re-run — do not raise it. **Verify the provider's current limits for your tenant**; they change and are not ours to assert.

Cutover window overruns

- **Likely cause:** Too much data is being moved at cutover because the **pre-stage** was skipped or its age filter was too recent, leaving a large delta.
- **Resolution:** Run a heavier **Pre-Stage Migration** (`BTW Pass – Pre-Stage – Per Wave`) well ahead of cutover with an age filter that leaves only recent items; confirm with Testing Test 3 that only the expected small delta remains before scheduling the DNS/MX cutover.

Items appear duplicated after a re-run

- **Likely cause:** A **full** pass was run where a **delta** pass was intended, or the pass configuration/age filter changed between runs.
- **Resolution:** Re-run as a **delta** pass so only items changed since pre-stage move; validate idempotency with Testing Test 7. **Verify pass (delta vs full) behavior for your version** in BitTitan's docs before expanding.

Recent mail/items missing after cutover

- **Likely cause:** The **delta pass** was not run at cutover, or coexistence routing delivered recent mail to the old platform after the user was declared migrated.
- **Resolution:** Run `BTW Pass – Delta Cutover – Per Wave`, then re-validate with `btw-postmigration-validation.py`. Confirm the split/dual-delivery routing rule is **keyed to the current migrated population** (a stale rule mis-delivers).

Documents migrate but SharePoint content lands in the wrong place

- **Likely cause:** A missing or wrong **SharePoint site** → **Shared Drive** mapping, or personal vs shared content routed incorrectly.
- **Resolution:** Fix the mapping in `BTW Project – Documents – Prod` and regenerate the line-item CSV with `btw-project-mapping-builder.py`; re-run the document pass for the affected items. Confirm target Shared Drives exist and the service identity has access.

Teams-to-Chat migration is incomplete or unexpected

- **Likely cause:** Teams-to-Chat coverage and behavior are **version/preview-dependent**; not all Teams data types map to Chat spaces.
- **Resolution:** Confirm exactly which collaboration data types are in scope in

`BTW Project – Teams to Chat – Prod`, and **verify Teams-to-Chat coverage for your version** in BitTitan's docs. Record what is migrated vs handled another way; validate the mapped Chat space manually (validation flags this as a manual check).

A line item won't resolve (source or target)

- **Likely cause:** The source UPN changed, or the target account is not provisioned/licensed yet.
- **Resolution:** Run `btw-project-mapping-builder.py --validate`; fix the source→target pair or

provision/license the target (per the identity source of truth), then re-import the line item. Do not start a pass with unresolved line items.

The Device Management Agent (DMA) was deployed but isn't needed (or is needed but wasn't)

- **Likely cause:** The DMA decision was skipped in design. Cloud-to-cloud mail typically needs no DMA;

some document/desktop-source paths do.

- **Resolution:** Revisit the **DMA decision** in the project design and **verify for your version and

project type** which paths require it. Remove speculative agents; deploy only where a project path genuinely requires one.

Mail is lost or double-delivered during coexistence

- **Likely cause:** The routing rule is not keyed to the migrated population, MX TTL was not lowered, or

split vs dual delivery is misconfigured.

- **Resolution:** Correct the coexistence design (`BTW Cutover – DNS/MX – Per Wave`): keep the routing

rule current, lower the MX TTL ahead of changes, and re-run Testing Test 5 (a message to a migrated and a not-yet-migrated user each reach the correct platform) before the DNS/MX cutover.

Validation fails on item counts even though data looks present

- **Likely cause:** The item-count **tolerance** is too tight, or expected counts include items the tool

legitimately skips (per BitTitan's skip rules).

- **Resolution:** Set a realistic tolerance in the design and confirm which item types the tool

skips/normalizes (**verify for your version**); re-run `btw-postmigration-validation.py` with the agreed tolerance. Investigate genuine gaps by re-queuing/retrying the specific line items.

Where to go for tool-internal issues

For MigrationWiz **console errors, per-error-code meanings, licensing questions, and support**, use **BitTitan's own documentation and support** — these are outside this solution's boundary. This guide resolves the **procedure, sequencing, endpoint-identity, throttling, pre-stage/delta, coexistence, and**

validation failures; the tool's internals are the vendor's.

7. Best Practices — BitTitan MigrationWiz: M365 → Google Migration Execution

Durable practices for executing an M365 → Google migration on MigrationWiz. They keep migrations predictable at enterprise scale and keep this solution inside its vendor boundary — the procedure is ours; the console internals and licensing are BitTitan's.

Prove endpoints before you move anything

The single highest-leverage step is proving both endpoints on **one test line item** before any bulk pass. A source app registration missing a consented permission or a target service account missing domain-wide delegation fails *every* line item identically — cheap to catch on one user, expensive to discover at scale. Re-prove endpoints whenever the app registration, service account, or consented scopes change.

Pre-stage heavy, delta light

Move the bulk of historical mail and documents in a **Pre-Stage Migration** well ahead of cutover, bounded by an age filter, so the **delta pass at cutover moves only recent changes**. This keeps the cutover window short and predictable. Fearing that a re-run duplicates items is the classic mistake — run correctly as a delta, a re-pass brings only what changed; validate idempotency (Testing Test 7) once and trust it. **Verify pass behavior for your version.**

Throttle deliberately; treat provider limits as real

Both sides throttle. Set concurrency from the planner's guidance, watch for sustained backoff with `btw-config-helper.py --check-throttle`, and lower concurrency rather than fighting the limit. A migration that crawls is usually one that tripped a protective limit and is backing off — not one that needs *more* threads. **Verify the current provider limits for your tenant**; they change.

One frozen design, run every wave

Freeze the endpoint design, the project/data-type/mapping design, the DMA decision, and the pre-stage/delta/throttle plan **before** the pilot, and run that same design unchanged every wave. Deciding per wave guarantees drift and rework. When the pilot exposes a defect, fix the **design**, not just the wave, so every later wave inherits the fix.

Validate per user, across every data type

A user is migrated only when mail **and** documents **and** chat are validated for them — never on mail alone. Keep the per-user validation record `BTW Validation – Mail/Docs/Chat – Per User` and let the gate, not optimism, decide the wave. Set an explicit item-count tolerance in the design so validation is objective.

Make the DMA a deliberate decision, not a reflex

Deploy the **Device Management Agent only where a project path actually requires it**. Cloud-to-cloud mail typically does not; some document/desktop-source paths do. Record "no DMA required" when that is the answer, so no one deploys agents speculatively. **Verify for your version and project type.**

Design coexistence up front; decommission it on purpose

Mail routing during the split is a first-class design (`BTW Cutover – DNS/MX – Per Wave`), not an afterthought: split delivery with the routing rule keyed to the migrated population, a lowered MX TTL for a fast back-out, and free/busy resolving across the divide. Keep the routing rule current as the migrated population grows, and **retire coexistence after the final wave** — never leave it running.

Every production change is reversible

Mail flow, DNS, sharing, and cutover are high-impact. Apply each under **change control** with a **documented, tested rollback**, and use the go/no-go gate to stop a wave that fails validation. A cutover with no proven back-out is an incident waiting to happen.

Stay inside the vendor boundary

Author and operate at the procedure level. For MigrationWiz **console screens, project-option internals, licensing, and support**, defer to **BitTitan's own documentation** and flag version-dependent behavior **"verify for your version."** Never invent a private BitTitan API — drive the tool with the **CSV line-item and validation artifacts** it consumes and produces (as the included Python tools do), and read Google/Microsoft only through their documented APIs.

Keep credentials out of code and least-privilege

The automation reads Google (Admin SDK / Directory / Reports) and Microsoft (Graph) **read-only** and uses **OAuth / a service account / ADC** — never hardcoded keys, never a committed key file. Scope the service account to exactly the read scopes the tools need; scope the migration's write identities to exactly what MigrationWiz's endpoints require. Rotate and revoke on completion.

8. Deliverables Summary — BitTitan MigrationWiz: M365 → Google Migration Execution

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The MigrationWiz execution model — projects, endpoints, agents, pre-stage/delta passes, coexistence — where it fits vs native tools, and the deployment model (the why)

Document	Purpose
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (MigrationWiz account + licensing, source app registration, target delegation, endpoint/project design, pre-stage/throttle plan, pilot + cutover runbook)
Global Requirements (Google)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: baseline, endpoint/project design, prove endpoints, pre-stage/delta + throttling plan, pilot wave, per-user validation, DNS/MX cutover, expand by wave, decommission coexistence
5. Testing Guide	Per-wave validation gates (endpoints, mappings, pre-stage, per-user mail/docs/chat, coexistence, throttling, delta idempotency, cutover) with pass/fail criteria
6. Troubleshooting Guide	Field failure modes and resolutions (auth errors, throttling, cutover overrun, duplication, missing items, mapping, DMA, coexistence)
7. Best Practices	Prove endpoints first, pre-stage heavy/delta light, throttle deliberately, validate per user, stay inside the vendor boundary

Named artifacts

Artifact	Name
Source/target scope & readiness baseline	`BTW Discovery – Source & Target – Baseline`
M365 source endpoint	`BTW Endpoint – M365 Source – Prod`
Google Workspace target endpoint	`BTW Endpoint – Google Target – Prod`
Mailbox migration project	`BTW Project – Mailbox – Prod`
Document migration project	`BTW Project – Documents – Prod`
Teams-to-Chat migration project	`BTW Project – Teams to Chat – Prod`
Line-item / user-mapping file	`BTW Mapping – User Pairs – Per Project`
Pilot wave	`BTW Wave – Pilot – Wave 0`
Per-wave production wave	`BTW Wave – <Segment> – Per Wave`
Pre-stage pass	`BTW Pass – Pre-Stage – Per Wave`
Delta / cutover pass	`BTW Pass – Delta Cutover – Per Wave`
Throttling profile	`BTW Throttle – Project Concurrency – Per Wave`
Per-user validation record	`BTW Validation – Mail/Docs/Chat – Per User`

Artifact	Name
DNS/MX cutover runbook	`BTW Cutover – DNS/MX – Per Wave`

Python tools

Script	What it does
`btw-readiness-assessment.py`	Read-only baseline of both sides — the M365 source over Microsoft Graph (users, mailboxes, OneDrive/SharePoint, Teams) and the Google target over the Admin SDK (users, licensing, service-account delegation presence) — reporting scope and blocking readiness gaps. No writes; no MigrationWiz API
`btw-project-mapping-builder.py`	Builds and validates the source→target **line-item CSV that MigrationWiz consumes** for each project (mailbox / documents / teamschat), checking resolvable source, provisioned+licensed target, and no duplicates/blanks. Writes only the CSV you name
`btw-wave-planner.py`	Splits the user list into balanced waves (small pilot first) and emits, per wave, the **pre-stage/delta pass schedule and throttling guidance** with advisory duration estimates. Dry-run/stdlib only; no credentials
`btw-postmigration-validation.py`	Per-user, read-only **post-migration validation** — confirms each target account exists/active and shows mail (Gmail), documents (Drive), and (manual) chat present within an item-count tolerance; emits the `BTW Validation – Mail/Docs/Chat – Per User` report. Read-only scopes
`btw-config-helper.py`	Endpoint-config + throttling-health helper: `--check-delegation` confirms the target service account's domain-wide delegation resolves the required scopes (and best-effort source app-registration reachability); `--check-throttle` samples the target API for throttling/backoff to tune the concurrency profile. Read-only

> *All five tools are **safe by default** — read-only or planning/dry-run — take **no hardcoded credentials** (OAuth / service account / ADC), commit **no keys**, and call **no private BitTitan API**. They drive the migration only via the **CSV line-item and validation artifacts** MigrationWiz documents as consuming/producing.*

The execution playbook (outline)

A single reusable runbook the migration runs from, assembled from the artifacts above:

1. **Baseline** — `BTW Discovery – Source & Target – Baseline`: source scope, target readiness, and

delegation gaps.

1. **Endpoint design** — `BTW Endpoint – M365 Source – Prod` + `BTW Endpoint – Google Target – Prod`:

identities and scopes, proven on one test line item.

1. **Project design** — `BTW Project – Mailbox/Documents/Teams to Chat – Prod`: data types, mappings,

and the explicit DMA decision.

1. **Pass plan** — `BTW Pass – Pre-Stage / Delta Cutover – Per Wave` + `BTW Throttle – Project

Concurrency – Per Wave`: pre-stage heavy, delta light, throttled within provider limits.

1. **Pilot wave** — `BTW Wave – Pilot – Wave 0`: shared mailbox, SharePoint site, and Team included.

2. **DNS/MX cutover runbook** — `BTW Cutover – DNS/MX – Per Wave`: readiness → pre-stage → coexistence

routing → delta cutover → validate → DNS/MX cutover → rollback.

1. **Per-user validation** — `BTW Validation – Mail/Docs/Chat – Per User`: mail **and** documents

and chat confirmed per user.

1. **Expand & decommission** — expand wave by wave behind the go/no-go gate, then retire coexistence

routing once the estate is fully on Gmail.

What the buyer receives

- This implementation guide and all supporting docs as a Word document and PDF.
- All five Python tools in a single ZIP.
- A repeatable, wave-based MigrationWiz execution following the model: **Assess → Design → Configure →

Pilot → Validate → Expand.**

Outcome

Every in-scope user moved from M365 to Google Workspace on MigrationWiz — mail, documents, and chat **validated per user**, the DNS/MX cutover **governed and reversible**, throttling controlled, and the whole migration **proven on a pilot wave** before any production wave — so migrations run predictably at enterprise scale, with the tool's console internals and licensing correctly deferred to BitTitan.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

btw-config-helper.py

btw-postmigration-validation.py

btw-project-mapping-builder.py

btw-readiness-assessment.py

btw-wave-planner.py